

Cooperative Gig Services Platform

Product Requirements Document (PRD) + Technical Design Document

SIH Problem Statement	26089
Title	Cooperative Gig Services Platform for Household & Community Services
Organization	Ministry of Cooperation
Department	National Council for Cooperative Training (NCCT)
Category	Software
Theme	Agriculture, FoodTech & Rural Development

Source basis: user-provided SIH problem statement. The architecture, stack, APIs and implementation roadmap in this document are proposed engineering decisions for the MVP.

Prepared for project planning, implementation and SIH prototype development

Contents

Part I. Product Requirements Document

1. Product Overview
2. Users & Stakeholders
3. Goals & Scope
4. Functional Requirements
5. Non-Functional Requirements
6. User Journeys
7. Success Metrics
8. Release Plan & Priorities

Part II. Technical Design Document

9. System Architecture
10. Technology Stack
11. Data Model
12. API Design
13. Worker Matching Engine
14. AI Demand Forecasting
15. Applications & Screens
16. Security
17. Notifications
18. Project Structure
19. Deployment
20. Development Roadmap
21. SIH Demo Scenario
22. Final System Summary

Part I - Product Requirements Document

1. Product Overview

The proposed platform creates a cooperative-owned digital marketplace connecting verified labour cooperative workers with households, businesses and institutions that require services. The source problem statement identifies service-provider registration and verification, skill profiling, booking, geo-based matching, payments, ratings, welfare/insurance, emergency booking, federation administration, multilingual support and AI-based demand forecasting/workforce allocation as expected solution areas.

Product objective

- Digitize cooperative worker availability and skills.
- Make service discovery and booking simple for customers.
- Increase trust through verification, certification and ratings.
- Support transparent booking, payment and invoicing.
- Help cooperatives monitor operations and allocate workforce using analytics and AI.

2. Users & Stakeholders

User	Primary needs
Customer	Discover trusted workers, choose a slot, book, pay, track service and provide feedback.
Worker	Create a profile, manage skills and availability, receive jobs, complete work and track earnings.
Cooperative Administrator	Verify workers, manage bookings, monitor operations and view cooperative analytics.
Federation Administrator	Manage multiple cooperatives, compare demand/coverage and support workforce planning.

3. Goals & Scope

3.1 MVP scope

- Authentication and role-based access.
- Worker registration, skills, certifications and verification.
- Service catalogue and customer discovery.
- Location-aware worker search and matching.
- Booking, scheduling and status tracking.
- Rating and feedback.
- Administrative dashboard and basic analytics.
- Emergency booking flow.
- Payment and invoice workflow (sandbox/mock for the initial prototype).
- Basic multilingual architecture.

3.2 Advanced scope

- AI demand forecasting by service and location.
- Workforce allocation recommendations.
- Welfare and insurance integration.
- Advanced geo-routing and operational analytics.

4. Functional Requirements

ID	Feature	Requirement
FR-01	Authentication	Customers, workers and admins can register/login and receive role-based access.
FR-02	Worker registration	Workers can maintain identity, service, skills, experience, certificates and cooperative membership.
FR-03	Worker verification	Cooperative admins can review and approve/reject worker profiles and certifications.
FR-04	Service search	Customers can select service, location, date/time and emergency priority.
FR-05	Worker matching	The platform ranks verified workers using skill match, distance, availability, rating and reliability.
FR-06	Booking	Customers can request, confirm, track and cancel bookings subject to status rules.
FR-07	Emergency booking	Urgent requests are prioritized for nearby available workers.
FR-08	Payment	The system records payment status and generates invoices.
FR-09	Rating	Completed services can be rated and reviewed.
FR-10	Welfare	Worker welfare/insurance status can be stored and monitored by authorized admins.
FR-11	Notifications	Booking, payment and service-status events generate notifications.
FR-12	Multilingual	The UI supports multiple languages through externalized strings.
FR-13	Admin dashboard	Admins can monitor users, bookings, workers and operational metrics.
FR-14	AI forecasting	The platform can forecast demand and expose recommendations in the admin dashboard.

5. Non-Functional Requirements

Area	Target / principle
Performance	Typical API requests should target sub-second responses under normal prototype load.
Security	HTTPS, secure password hashing, JWT/session controls, authorization and server-side validation.
Scalability	Service modules should be separable so the prototype can grow from a small cooperative to federation-scale usage.

Area	Target / principle
Usability	Customer booking should be a short, clear flow with visible verification and status.
Availability	Critical booking workflows should degrade gracefully if a non-critical subsystem is unavailable.
Maintainability	Use typed contracts, modular backend services and versioned APIs.

6. User Journeys

Customer journey

Open app → choose location → choose service → browse verified workers → select worker → choose date/time → confirm booking → payment → service → rating

Worker journey

Register → submit profile → cooperative verification → add skills → set availability → receive request → accept → perform service → complete job → earnings/history

Admin journey

Login → dashboard → verify workers → monitor bookings → review analytics → inspect demand forecast → allocate workforce → monitor outcomes

7. Success Metrics

Metric group	Examples
Customer	Booking completion rate, repeat usage, average booking time, rating.
Worker	Verified workers, active workers, jobs completed, utilization, average rating.
Cooperative	Bookings, revenue/transaction value, fulfillment rate, active worker coverage.
AI	Forecast accuracy, reduction in unassigned demand, utilization improvement.

8. Release Plan & Priorities

P0 - Must have

- Authentication
- Worker registration and verification
- Services and worker discovery
- Geo-matching
- Booking and status
- Admin dashboard
- Ratings

P1 - Important

- Payments and invoices

- Emergency booking
- Notifications
- Multilingual UI

P2 - Advanced

- AI demand forecasting
- Workforce allocation
- Insurance integration
- Advanced analytics

Part II - Technical Design Document

9. System Architecture

The proposed MVP uses a client-server architecture with a shared backend API, relational data store, geo-spatial search and a separate AI service. This keeps the marketplace usable independently from forecasting while allowing AI to consume historical booking data.

Customer App Worker App Admin Dashboard

\\ /

API Gateway

|

+-----+-----+

|||

Auth/Users Booking Worker/Matching

|||

+-----+-----+

|
PostgreSQL + PostGIS

|

+-----+-----+

||

Redis Payment/Notifications

|

+-----+-----+

|
AI Service

|

Demand Forecast

10. Recommended Technology Stack

Layer	Recommendation	Purpose
Mobile	Flutter	Cross-platform customer/worker app.
Web admin	React + TypeScript	Federation/cooperative dashboard.
Backend	Node.js + Express + TypeScript	REST API and business logic.
Database	PostgreSQL + PostGIS	Transactional data and spatial queries.
Cache	Redis	Availability, caching, transient booking locks.
Notifications	Firebase Cloud Messaging	Push/in-app notifications.
AI	Python + FastAPI	Forecasting/inference service.
Deployment	Docker + cloud VM/container platform	Reproducible deployment and scaling.

11. Data Model

Core entities

Entity	Key fields
users	id, name, phone/email, password_hash, role, language, timestamps

Entity	Key fields
workers	id, user_id, cooperative_id, experience, verification_status, lat, long, availability_status, rating
skills	id, name, category, description
worker_skills	worker_id, skill_id, experience_level, certificate_id
certifications	worker_id, issuer, certificate_number, validity, verification_status, document_url
cooperatives	id, name, registration_number, location, contact, status
services	id, name, category, description, base_price, emergency_available
bookings	customer, worker, service, cooperative, schedule, coordinates, status, priority, pricing
payments	booking_id, customer_id, worker_id, amount, method, transaction_id, status
ratings	booking_id, customer_id, worker_id, rating, comment
worker_welfare	worker_id, insurance/welfare fields, expiry, status

12. API Design

Authentication

POST /api/auth/register
 POST /api/auth/login
 POST /api/auth/logout
 GET /api/auth/me

Workers and services

POST /api/workers
 GET /api/workers/:id
 PUT /api/workers/:id
 PATCH /api/workers/:id/availability
 POST /api/workers/:id/skills
 GET /api/services

Matching and bookings

GET /api/matching/workers
 POST /api/bookings
 GET /api/bookings
 GET /api/bookings/:id
 PATCH /api/bookings/:id/status

Payments, ratings and admin

POST /api/payments/create
 POST /api/payments/verify
 POST /api/ratings
 GET /api/workers/:id/ratings
 GET /api/admin/dashboard
 PATCH /api/admin/workers/:id/verify
 GET /api/admin/analytics

13. Worker Matching Engine

The matching engine should first reduce the candidate pool and then rank the remaining verified workers. A practical prototype score is: 35% skill match, 25% distance, 20% availability, 10% rating and 10% reliability. These weights are proposed and should be tuned with real booking data.

```
Request
↓
Filter by required skill
↓
Filter verified workers
↓
Filter availability
↓
Search within radius using PostGIS
↓
Calculate matching score
↓
Rank workers
↓
Notify highest-ranked candidates
```

Emergency requests should increase the weight of proximity, availability and response time.

14. AI Demand Forecasting

The AI service is designed as an independent module that consumes historical booking data and returns forecasts by service, location and time period. The prototype should start with interpretable tabular/time-series approaches and be evaluated using validation data before selecting the final model.

```
Historical bookings
↓
Cleaning + feature engineering
↓
Training/validation
↓
Forecast model
↓
Inference API
↓
Admin dashboard
```

Example features

- Date/day of week
- Time period
- Location/zone
- Service category
- Historical request count
- Emergency request count
- Seasonality or holiday indicators, where available

Example output

Zone: Coimbatore North
Service: Plumbing
Forecast: High demand
Expected requests: 32 (example output for prototype demo)

Workforce allocation example

Predicted demand = 40 electricians

Available = 25

Gap = 15

Recommendation: schedule/reallocate additional coverage to the zone/time window

15. Applications & Screens

Customer app

- Splash / onboarding
- Login / registration
- Home + service categories
- Service details
- Nearby verified workers
- Worker profile
- Booking and scheduling
- Payment
- Booking tracking
- Invoice
- Rating

Worker app

- Login
- Dashboard
- Availability
- Incoming requests
- Booking details
- Navigation/service status
- Job completion
- Earnings
- Profile and certifications

Admin dashboard

- KPI cards: workers, verified workers, bookings, transaction value
- Booking analytics
- Worker distribution map
- Verification queue
- AI demand forecast
- Workforce allocation recommendations

16. Security

- Use HTTPS in all deployed environments.

- Hash passwords using a strong password hashing algorithm.
- Use JWT or secure server sessions with expiration and refresh strategy as appropriate.
- Enforce role-based access control at API level.
- Validate and sanitize user inputs.
- Protect payment/webhook endpoints from spoofing and replay.
- Keep personally identifiable and sensitive welfare data accessible only to authorized roles.
- Log security-relevant actions such as verification and administrative status changes.

17. Notification Architecture

Booking Created → Notification Service → Push/In-app
 Booking Accepted → Notification Service → Customer
 Worker Arrival → Notification Service → Customer
 Payment Success → Notification Service → Customer/Worker
 Job Completed → Notification Service → Customer/Admin

18. Project Structure

```
mobile/
lib/core/
lib/models/
lib/services/
lib/providers/
lib/screens/auth/
lib/screens/customer/
lib/screens/worker/
lib/widgets/
backend/
src/controllers/
src/services/
src/models/
src/routes/
src/middleware/
src/utils/
src/config/
ai-service/
data/
preprocessing/
training/
inference/
```

19. Deployment

For the SIH prototype, keep deployment simple enough for the team to operate. A Dockerized backend, managed PostgreSQL/PostGIS instance and a separately deployed AI API are sufficient.

```
Mobile/Web
↓
Cloud / reverse proxy
↓
Backend API containers
├── PostgreSQL + PostGIS
├── Redis
├── Object storage for documents
└── AI inference service
```

20. Development Roadmap

Week	Deliverables
Week 1	Project setup, database schema, authentication, roles, base UI
Week 2	Worker profile, skills, certifications, verification, availability
Week 3	Service catalogue, customer discovery, location capture, worker search
Week 4	Matching, booking, scheduling, notifications
Week 5	Payment sandbox, invoices, transaction history
Week 6	Ratings, worker earnings, admin dashboard and analytics
Week 7	AI dataset pipeline, forecasting model, prediction API
Week 8	Integration testing, UI polish, deployment and SIH demo preparation

21. SIH Demo Scenario

The strongest demo should follow one end-to-end story rather than showing isolated screens.
Example: an emergency plumbing request from a customer.

```

Customer selects Emergency Plumbing
↓
Location captured
↓
Nearby verified plumbers retrieved
↓
Workers ranked by skill + distance + availability + rating
↓
Top worker receives request
↓
Worker accepts
↓
Customer tracks status
↓
Job completed
↓
Digital payment recorded + invoice generated
↓
Customer rates worker
↓
Admin dashboard updates
↓
Booking becomes part of historical demand data

```

22. Final System Summary

```

CUSTOMER APP └─
WORKER APP └─┬─> BACKEND API ─> POSTGRESQL/POSTGIS
ADMIN WEB └─┬─
├─> REDIS
├─> PAYMENTS
├─> NOTIFICATIONS
├─> AI SERVICE
├─
├─ Demand Forecasting
├─ Workforce Allocation

```

The core design principle is to make the marketplace fully functional without AI, then use accumulated booking data to strengthen forecasting and workforce planning. This reduces demo risk while preserving the advanced capability required by the problem statement.

Implementation note

This document distinguishes the source problem statement from the proposed engineering solution. Technology choices, scoring weights, API paths, development weeks and example numbers are implementation recommendations, not requirements stated by the SIH source document.